DATABASE MANAGEMENT SYSTEM

MINI PROJECT REVIEW

Project Title: Social Media Analytics Engine

Submitted by

Group Id: 5

Div: 13

Batch: 2

Team Members:

Sr.No	*PRN*	*Name of the students*
1	1262252008	Anushree Singh
2	1262252785	Chinmayi Gadre
3	1262252045	Pournami Roy
4	1262252891	Palak Soni
5	1262253274	Tanishka Tiwari

Problem Statement:

Social media platforms generate massive amounts of data through posts, comments, likes, shares, and user interactions. Analyzing this data to identify trending posts, measure user engagement, evaluate creator performance, and track activity patterns is challenging. The Social Media Analytics Engine aims to address these challenges by developing a MySQL-based database system that efficiently manages social media data, performs advanced analytical queries, maintains data integrity, and provides meaningful insights through an interactive dashboard.

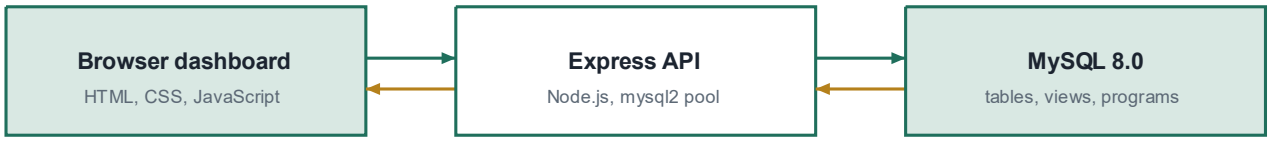
Social Media Analytics Engine

DBMS Mini Project Report: ER diagram, schema, DDL, DML, stored programs and front-end

Project overview

The Social Media Analytics Engine is a relational database system that stores social-media-style activity (users, posts, comments, follows and engagement events) and analyses it with advanced SQL. It answers questions such as which posts are trending, which creators perform best, how activity changes through the day, and whether an index makes a query faster. The database is the academic focus; the web dashboard exists to show the database working.

The system is built on **MySQL 8.0** (InnoDB), with a **Node.js + Express** API using the **mysql2** driver, and a browser dashboard written in plain HTML, CSS and JavaScript. On first start it creates its own schema and generates a dataset of 600 users, 6,000 posts, 24,000 comments, 14,000 follows and roughly 177,000 interactions.



What the database contains

Object	Count	Items
Core tables	5	users, posts, comments, followers, interactions
Supporting tables	4	interaction_audit, post_audit, user_engagement_report, trending_report
Views	3	v_post_engagement, v_user_engagement, v_daily_engagement
Secondary indexes	9	on timestamps, post type, topic and country (foreign keys are indexed by InnoDB itself)
Stored functions	3	CalculatePostEngagement, CalculateUserEngagement, CalculateWindowEngagement
Stored procedures (cursors)	2	GenerateUserEngagementReport, GenerateTrendingPostReport
Triggers	3	trg_no_self_engagement, trg_interaction_audit, trg_post_delete_audit
Analytical queries	14	joins, CTEs, window functions, subqueries, anti-join, self join

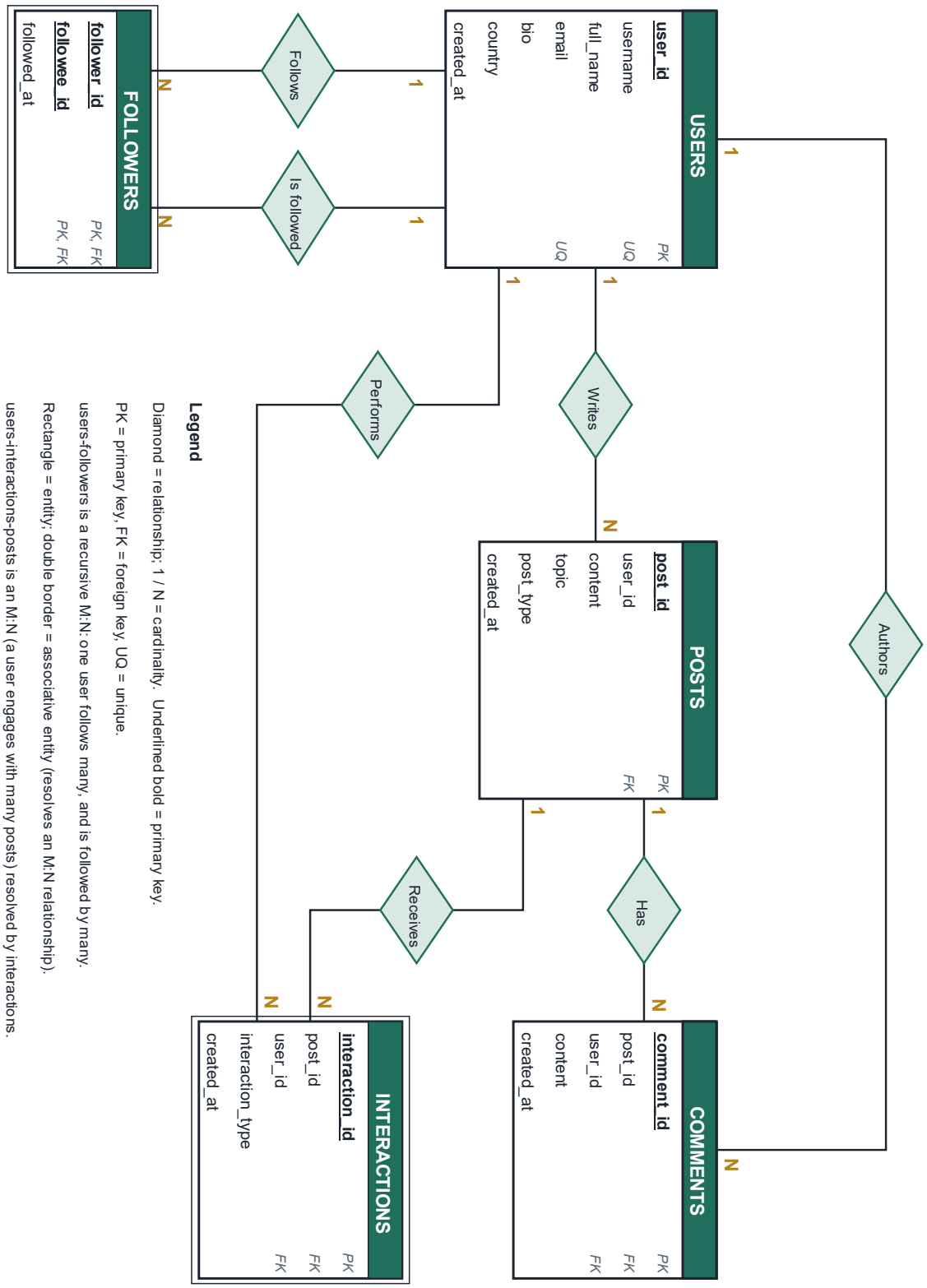
Engagement scoring used throughout

Every engagement figure is derived, never stored. Weights: **view 0.2, like 1, save 2, comment 2, share 3**. A share is worth fifteen views because it puts the post in front of a new audience.

Contents

1. ER diagram
2. Schema diagram
3. DDL statements (also supplied as social_analytics_ddl.txt)
4. DML queries, stated in English, with the front-end and back-end flow
5. Stored programs: procedures with cursors, functions and triggers, with scenarios
6. Front-end and back-end connectivity with MySQL

1. ER Diagram



Legend

Diamond = relationship; 1 / N = cardinality. Underlined bold = primary key.

PK = primary key, FK = foreign key, UQ = unique.

users-followers is a recursive M:N: one user follows many, and is followed by many.

Rectangle = entity; double border = associative entity (resolves an M:N relationship).

users-interactions-posts is an M:N (a user engages with many posts) resolved by interactions.

1.1 Entities and relationships

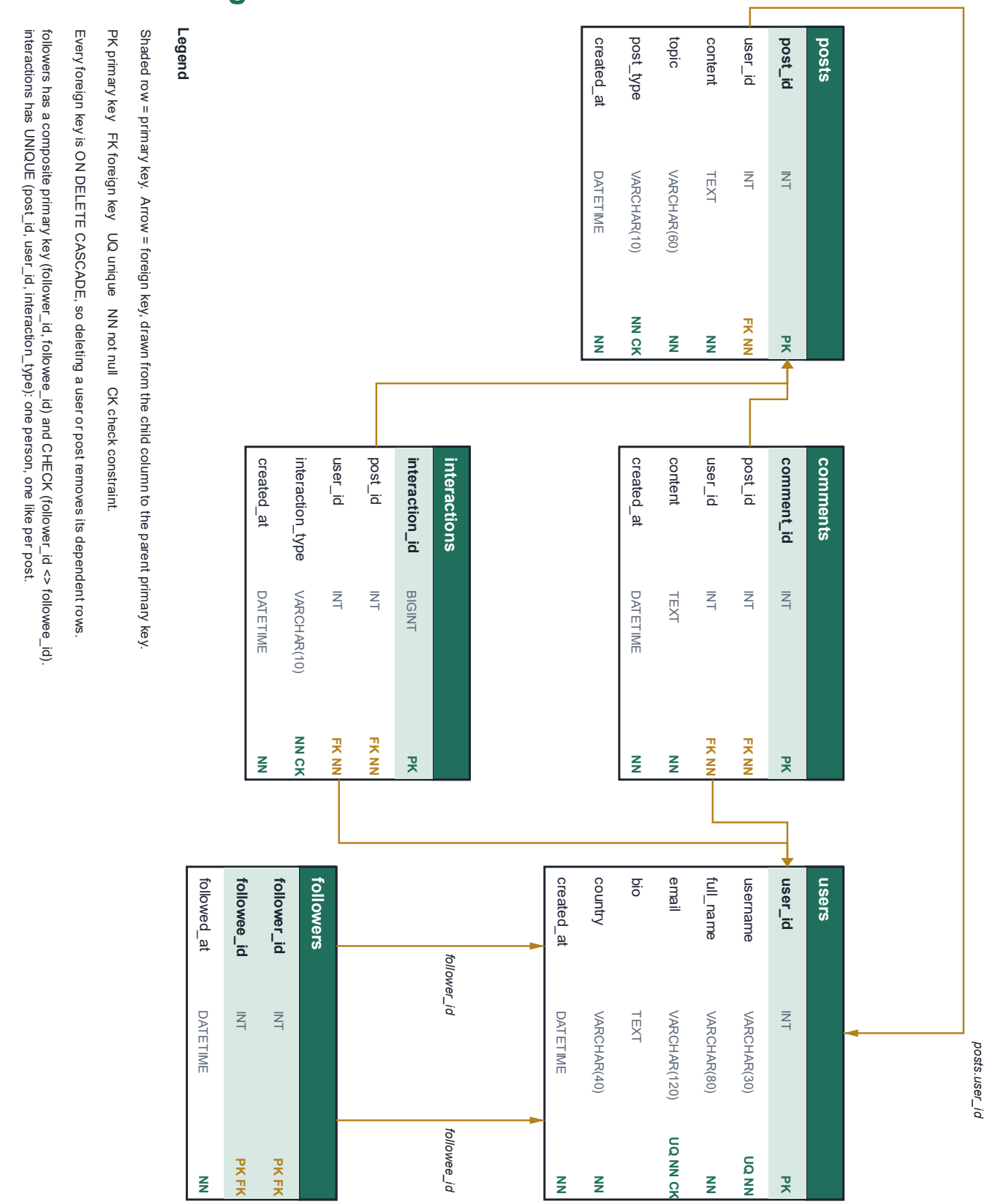
The model has five entities. Two of them, **interactions** and **followers**, are associative entities that resolve many-to-many relationships.

Relationship	Cardinality	How it is implemented
A user writes posts	1 : N	posts.user_id references users.user_id
A post has comments	1 : N	comments.post_id references posts.post_id
A user writes comments	1 : N	comments.user_id references users.user_id
A user engages with posts (like, share, save, view)	M : N	interactions resolves it. It carries its own attributes (interaction_type, created_at), so it is an associative entity with its own surrogate key.
A user follows users	M : N, recursive	followers(follower_id, followee_id). The composite primary key stops a duplicate follow and a CHECK stops a self-follow.

1.2 Design decisions

- **Third normal form.** posts stores user_id only, never the author's username or country, so renaming a user cannot leave stale copies. Every non-key attribute depends on the key, the whole key, and nothing but the key.
- **Derived data is not stored.** There is no like_count on posts and no follower_count on users. They are computed by views, so the data cannot disagree with itself.
- **interactions is a separate fact table.** It is high volume (about 177,000 rows) and each row is an independent event with its own user, post, type and timestamp.
- **comments is separate from interactions.** A comment has text of its own and a different weight in the score.

2. Schema Diagram



Legend

Shaded row = primary key. Arrow = foreign key, drawn from the child column to the parent primary key.

PK primary key FK foreign key UQ unique NN not null CK check constraint.

Every foreign key is ON DELETE CASCADE, so deleting a user or post removes its dependent rows.

followers has a composite primary key (follower_id, followee_id) and a CHECK (follower_id <> followee_id). interactions has UNIQUE (post_id, user_id, interaction_type): one person, one like per post.

2.1 Table and row level constraints

Table	Constraints
users	PRIMARY KEY (user_id) AUTO_INCREMENT; UNIQUE (username); UNIQUE (email); CHECK (email LIKE '%@%.%'); NOT NULL on every column except bio
posts	PRIMARY KEY (post_id); FOREIGN KEY (user_id) to users ON DELETE CASCADE; CHECK (post_type IN ('text','image','video','link')); NOT NULL
comments	PRIMARY KEY (comment_id); FOREIGN KEY (post_id) to posts and FOREIGN KEY (user_id) to users, both ON DELETE CASCADE; NOT NULL
followers	Composite PRIMARY KEY (follower_id, followee_id); both columns FOREIGN KEY to users ON DELETE CASCADE; CHECK (follower_id <> followee_id)
interactions	PRIMARY KEY (interaction_id) BIGINT; FOREIGN KEYs to posts and users, both CASCADE; CHECK (interaction_type IN ('like','share','save','view')); UNIQUE (post_id, user_id, interaction_type)

Constraint types demonstrated: PRIMARY KEY (single and composite), FOREIGN KEY with a referential action, UNIQUE (single and composite), CHECK (value list, pattern and column comparison), NOT NULL, AUTO_INCREMENT and DEFAULT (on the audit tables).

2.2 Supporting tables

Table	Written by	Purpose
interaction_audit	trg_interaction_audit	One row for every interaction inserted, with the moment it arrived
post_audit	trg_post_delete_audit	A record of each deleted post, since its comments and interactions disappear with it
user_engagement_report	GenerateUserEngagementReport	Creator-level summary: posts counted, total, best post, average
trending_report	GenerateTrendingPostReport	Posts that earned points inside a chosen window, rebuilt on every run

3. DDL Statements

The complete script is supplied as one text file, `social_analytics_ddl.txt`, for the group submission. It is reproduced here. Run order: core tables, supporting tables, indexes, views. Foreign keys require the parent table to exist first, which is why users comes before posts.

Part A: core tables

```
CREATE TABLE users (       user_id      INT AUTO_INCREMENT
PRIMARY KEY,      username    VARCHAR(30)  NOT NULL UNIQUE,
full_name  VARCHAR(80)  NOT NULL,     email      VARCHAR(120)
NOT  NULL UNIQUE,        bio            TEXT,        country
VARCHAR(40)  NOT NULL,     created_at DATETIME     NOT NULL,
CONSTRAINT users_email_shape CHECK (email LIKE '%@%.%')
) ENGINE = InnoDB;

CREATE TABLE posts (     post_id     INT AUTO_INCREMENT PRIMARY KEY,     user_id     INT
NOT NULL,     content     TEXT        NOT NULL,     topic      VARCHAR(60) NOT NULL,
post_type  VARCHAR(10) NOT NULL,     created_at DATETIME     NOT NULL,     CONSTRAINT
posts_author_fk FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT posts_type_allowed CHECK (post_type IN ('text', 'image', 'video', 'link'))
) ENGINE = InnoDB;

CREATE TABLE comments (     comment_id INT AUTO_INCREMENT PRIMARY KEY,     post_id     INT      NOT
NULL,     user_id     INT      NOT NULL,     content     TEXT     NOT NULL,     created_at DATETIME
NOT NULL,     CONSTRAINT comments_post_fk FOREIGN KEY (post_id) REFERENCES posts (post_id) ON
DELETE CASCADE,     CONSTRAINT comments_author_fk FOREIGN KEY (user_id) REFERENCES users (user_id)
ON DELETE CASCADE
) ENGINE = InnoDB;
```

```
-- Many-to-many relationship of users with themselves.
-- The composite primary key is what stops a duplicate
follow. CREATE TABLE followers (     follower_id INT      NOT
NULL,     followee_id INT      NOT NULL,     followed_at
DATETIME NOT NULL,     PRIMARY KEY (follower_id,
followee_id),
    CONSTRAINT followers_follower_fk FOREIGN KEY (follower_id) REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT followers_followee_fk FOREIGN KEY (followee_id) REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT followers_no_self_follow CHECK (follower_id <> followee_id)
) ENGINE = InnoDB;

-- The high-volume fact table. One row per engagement event.
CREATE TABLE interactions (
    interaction_id   BIGINT AUTO_INCREMENT PRIMARY KEY,     post_id          INT         NOT NULL,
user_id          INT         NOT NULL,     interaction_type VARCHAR(10) NOT NULL,     created_at
DATETIME    NOT NULL,     CONSTRAINT interactions_post_fk FOREIGN KEY (post_id) REFERENCES posts
(post_id) ON DELETE CASCADE,
    CONSTRAINT interactions_user_fk FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT interactions_type_allowed CHECK (interaction_type IN ('like', 'share', 'save', 'view')),
    CONSTRAINT interactions_one_per_user_per_type UNIQUE (post_id, user_id, interaction_type)
) ENGINE = InnoDB;
```

Part B: supporting tables

```
CREATE TABLE IF NOT EXISTS interaction_audit (
audit_id         BIGINT AUTO_INCREMENT PRIMARY KEY,
interaction_id   BIGINT      NULL,     post_id
INT         NOT NULL,     user_id          INT
NOT NULL,     interaction_type VARCHAR(10) NOT NULL,
action           VARCHAR(10) NOT NULL,
    recorded_at      DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
INDEX idx_audit_recorded (recorded_at)
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS post_audit (
audit_id    BIGINT AUTO_INCREMENT PRIMARY KEY,
post_id     INT         NOT NULL,     user_id
INT         NOT NULL,
    topic       VARCHAR(60) NOT NULL,     action
VARCHAR(10) NOT NULL,     recorded_at DATETIME    NOT NULL
DEFAULT CURRENT_TIMESTAMP ) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS user_engagement_report
(     user_id          INT PRIMARY KEY,
username         VARCHAR(30)    NOT NULL,
posts_counted    INT            NOT NULL,
total_points     DECIMAL(12, 1) NOT NULL,
best_post_id     INT            NULL,
best_post_points DECIMAL(10, 1) NULL,
average_points   DECIMAL(10, 1) NOT NULL,
generated_at     DATETIME       NOT NULL ) ENGINE
= InnoDB;

CREATE TABLE IF NOT EXISTS trending_report (      report_row    INT AUTO_INCREMENT
PRIMARY KEY,     post_id      INT           NOT NULL,     author       VARCHAR(30)
NOT NULL,     topic        VARCHAR(60)    NOT NULL,     points       DECIMAL(10, 1)
NOT NULL,   -- earned inside the window, not lifetime       window_start DATETIME
NOT NULL,      window_end    DATETIME         NOT NULL,      generated_at DATETIME
NOT NULL ) ENGINE = InnoDB;
```

Part C: secondary indexes

```
CREATE INDEX idx_interactions_created_at   ON interactions (created_at);
CREATE INDEX idx_interactions_type_created ON interactions (interaction_type, created_at);
CREATE INDEX idx_posts_created_at          ON posts (created_at);
CREATE INDEX idx_posts_type                ON posts (post_type);
CREATE INDEX idx_posts_topic               ON posts (topic);
CREATE INDEX idx_comments_created_at       ON comments (created_at);
CREATE INDEX idx_followers_followed_at     ON followers (followed_at);
CREATE INDEX idx_users_country             ON users (country);
CREATE INDEX idx_users_created_at          ON users (created_at);
```

Part D: views

```
-- One row per post, with every engagement metric derived on the fly.
CREATE VIEW v_post_engagement AS
SELECT p.post_id,
       p.user_id AS author_id,
       u.username AS author,
       p.topic,
       p.post_type,
```

```
       p.created_at AS posted_at,        SUM(CASE WHEN i.interaction_type =
'like' THEN 1 ELSE 0 END) AS likes,
       SUM(CASE WHEN i.interaction_type = 'share' THEN 1 ELSE 0 END) AS shares,
       SUM(CASE WHEN i.interaction_type = 'save' THEN 1 ELSE 0 END) AS saves,
       SUM(CASE WHEN i.interaction_type = 'view' THEN 1 ELSE 0 END) AS views,
       (SELECT COUNT(*) FROM comments c WHERE c.post_id = p.post_id) AS comments,
       COUNT(i.interaction_id) AS total_interactions,
       COUNT(DISTINCT i.user_id) AS unique_engagers,
       MAX(i.created_at) AS last_interaction_at,
       ROUND(SUM(CASE WHEN i.interaction_type = 'view' THEN 0.2
                      WHEN i.interaction_type = 'like' THEN 1.0
                      WHEN i.interaction_type = 'save' THEN 2.0
                      WHEN i.interaction_type = 'share' THEN 3.0
                      ELSE 0 END)
             + (SELECT COUNT(*) FROM comments c WHERE c.post_id = p.post_id) * 2.0, 1) AS engagement_score
FROM posts p
JOIN users u ON u.user_id = p.user_id
LEFT JOIN interactions i ON i.post_id = p.post_id
GROUP BY p.post_id, p.user_id, u.username, p.topic, p.post_type, p.created_at;

-- One row per user: what they publish and how well it
lands. CREATE VIEW v_user_engagement AS SELECT u.user_id,
       u.username,
       u.full_name,
       u.country,
       u.created_at AS joined_at,
COUNT(pe.post_id) AS posts,
       COALESCE(SUM(pe.engagement_score), 0) AS engagement_score,
       ROUND(COALESCE(AVG(pe.engagement_score), 0), 1) AS avg_score_per_post,
       COALESCE(SUM(pe.likes), 0) AS likes_received,
       COALESCE(SUM(pe.comments), 0) AS comments_received,
       (SELECT COUNT(*) FROM followers f WHERE f.followee_id = u.user_id) AS followers,
(SELECT COUNT(*) FROM followers f WHERE f.follower_id = u.user_id) AS following
FROM users u
LEFT JOIN v_post_engagement pe ON pe.author_id = u.user_id
GROUP BY u.user_id, u.username, u.full_name, u.country, u.created_at;
-- One row per calendar day of platform activity.
CREATE VIEW v_daily_engagement AS
SELECT DATE(created_at) AS activity_date,
       COUNT(*) AS interactions,
       SUM(CASE WHEN interaction_type = 'like' THEN 1 ELSE 0 END) AS likes,
       SUM(CASE WHEN interaction_type = 'share' THEN 1 ELSE 0 END) AS shares,
       SUM(CASE WHEN interaction_type = 'save' THEN 1 ELSE 0 END) AS saves,
       SUM(CASE WHEN interaction_type = 'view' THEN 1 ELSE 0 END) AS views,
       COUNT(DISTINCT user_id) AS active_users,
       COUNT(DISTINCT post_id) AS posts_touched
FROM interactions
GROUP BY DATE(created_at);
```

4. DML Queries

Each requirement below is written as the English question a user would ask. The SQL itself lives on the server, in `queries/catalog.js`. The browser never sends SQL to the server for these: it sends the query id, the server looks up and runs the statement, and the dashboard shows the rows together with the SQL that produced them.

4.1 How a requirement becomes a result on screen

- **Front-end.** The user opens the **Query lab** page and selects a query from the list. The page shows the question, the SQL concepts used, and the SQL text.
- **Request.** Pressing run sends `POST /api/queries/:id/run` to the Express API.
- **Back-end.** The API finds the statement by id in the catalog, runs it through the mysql2 connection pool, and measures how long MySQL took.
- **Response.** Rows, column names and elapsed time return as JSON, and the page draws them as a table.

4.2 Analytical queries (14)

#	Query lab entry	Requirement in English	SQL techniques
1	**Highest engagement posts**	List the 20 posts with the most interactions (at least 40 each), showing likes, shares, saves, total interactions and how many different people engaged.	JOIN, GROUP BY, HAVING, conditional SUM

2	**Creator leaderboard**	Rank creators by total weighted engagement, overall and within their own country, and give each a percentile.	CTE, RANK, DENSE_RANK, PERCENT_RANK, PARTITION BY
3	**Trending in the last 7 days**	Show the posts collecting the most activity in the last seven days, whenever they were published, and how old each post is.	CTE, UNION ALL, weighted SUM, DENSE_RANK
4	**Who has been active in the last 3 days**	Find the people who performed the most actions in the last three days, with their likes, shares and the number of posts they touched.	Narrow date filter, JOIN, GROUP BY
5	**Post performance against its author**	For authors with at least eight posts, compare every post with the average of that author's own posts and label it breakout, above average or below average.	Window AVG, PARTITION BY, CASE, CTE
6	**Daily activity with a 7 day moving average**	Show daily interaction totals for the last 45 days with a seven-day moving average, the change from yesterday and a running total.	ROWS BETWEEN frame, LAG, running SUM
7	**Each creator's best three posts**	For every creator with at least ten posts, show their three highest-scoring posts.	ROW_NUMBER, PARTITION BY, filter on window result
8	**Users above the average follower count**	Find users who have more followers than the average account, with their followers, following and post counts.	Correlated subquery, scalar subquery, derived table
9	**When the platform is awake**	Show interactions by hour of the day and each hour's share of all activity.	HOUR(), GROUP BY, SUM() OVER ()
10	**Accounts that never engage**	List users who registered but have never liked, shared, saved or viewed anything, and how many posts they wrote.	NOT EXISTS anti-join
11	**Interaction mix of the busiest posts**	For the ten busiest posts, show how engagement splits between views, likes, saves and shares, as a percentage of each post.	CTE, window SUM over partition
12	**Mutual connections**	List pairs of users who follow each other, and when each follow happened.	Self join on a composite key
13	**Which format performs best**	Compare text, image, video and link posts by number published, average interactions, average score and best single post.	Aggregates, GROUP BY, ratio
14	**Follower growth month by month**	Show new follows per month, the cumulative total, the change from the previous month and each month's share.	DATE_FORMAT, running SUM, LAG

4.3 Search and data-changing operations

These cover the remaining DML: parameterised SELECT searches, and INSERT and DELETE statements that fire the triggers described in section 5.

Requirement in English	Statement	Where the user does it
Search users by name or username and sort them by engagement, followers, posts or name.	SELECT with LIKE and ? placeholders on v_user_engagement; the sort column is checked against an allow-list	Explore data, Users tab (GET /api/users)
Search posts by text and filter by format.	SELECT with LIKE and ? placeholders on v_post_engagement	Explore data, Posts tab (GET /api/posts)

Publish a new post.	INSERT INTO posts (user_id, content, topic, post_type, created_at) VALUES (?, ?, ?, ?, NOW())	Procedures and triggers page (POST /api/posts)
React to a post as a user.	INSERT INTO interactions (post_id, user_id, interaction_type, created_at) VALUES (?, ?, ?, NOW()); the audit trigger fires and the integrity trigger can refuse it	Like buttons (POST /api/interactions)
Delete a post and everything attached to it.	DELETE FROM posts WHERE post_id = ?; comments and interactions cascade and the delete-audit trigger fires	Delete button (DELETE /api/posts/:id)
Run my own read-only query.	A single SELECT or WITH, inside a read-only transaction with a timeout, capped at 200 rows	SQL console (POST /api/sql)

Because every value reaches MySQL through a placeholder, user text is never joined into SQL text, which prevents SQL injection.

5. Stored Programs: Procedures, Functions and Triggers

The procedural layer is written in MySQL's stored-program language. It is the MySQL counterpart of Oracle PL/SQL: `CREATE PROCEDURE`, `CREATE FUNCTION`, `CREATE TRIGGER`, `DECLARE ... CURSOR`, handlers and `SIGNAL SQLSTATE` in place of RAISE_APPLICATION_ERROR. The review sheet says PL/SQL while also requiring MySQL connectivity, so the team should confirm with the faculty that MySQL's dialect is acceptable.

Kind	Name	Fires or is called
Function	CalculatePostEngagement(post_id)	Called by procedures, queries and the API
Function	CalculateUserEngagement(user_id)	Called by the API and from SQL
Function	CalculateWindowEngagement(post_id, start, end)	Called by the trending procedure
Procedure + cursor	GenerateUserEngagementReport(user_id)	CALL from the API
Procedure + cursor	GenerateTrendingPostReport(start, end, limit)	CALL from the API
Trigger	trg_no_self_engagement	BEFORE INSERT on interactions
Trigger	trg_interaction_audit	AFTER INSERT on interactions
Trigger	trg_post_delete_audit	AFTER DELETE on posts

5.1 Procedures with cursors

Procedure 1: GenerateUserEngagementReport(IN p_user_id INT)

Scenario	A creator opens their analytics page and asks for a summary of their whole catalogue.
Cursor use	A cursor over every post the creator owns is opened and fetched one post_id at a time. For each row the procedure calls CalculatePostEngagement, adds to a running total, counts the post and remembers the best performer so far. A CONTINUE HANDLER FOR NOT FOUND ends the loop.
Output	The summary is written to user_engagement_report with REPLACE INTO and selected back to the caller.
Why a cursor	It carries state between rows (running total, count, best so far) and calls a function on each row.
Error handling	An unknown user id raises SIGNAL SQLSTATE '45000' with the message "No user with that id".

Demonstrated at	Procedures and triggers page, button GenerateUserEngagementReport

```
CREATE PROCEDURE GenerateUserEngagementReport(IN p_user_id INT)
BEGIN
    DECLARE v_done          INT DEFAULT 0;
    DECLARE v_post_id       INT;
    DECLARE v_points        DECIMAL(10, 1);
    DECLARE v_total         DECIMAL(12, 1) DEFAULT 0;
    DECLARE v_count         INT DEFAULT 0;
    DECLARE v_best_post     INT DEFAULT NULL;
DECLARE v_best_points   DECIMAL(10, 1) DEFAULT 0;
    DECLARE v_username      VARCHAR(30);

    DECLARE post_cursor CURSOR FOR
        SELECT post_id FROM posts WHERE user_id = p_user_id ORDER BY post_id;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    SELECT username INTO v_username FROM users WHERE user_id = p_user_id;
    IF v_username IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No user with that id';
    END IF;

    OPEN post_cursor;     read_loop:
LOOP         FETCH post_cursor INTO
v_post_id;
        IF v_done = 1 THEN
LEAVE read_loop;
        END IF;

        SET v_points = CalculatePostEngagement(v_post_id);
        SET v_total = v_total + v_points;
        SET v_count = v_count + 1;

        IF v_points > v_best_points THEN
            SET v_best_points = v_points;
            SET v_best_post = v_post_id;
        END IF;
    END LOOP;
    CLOSE post_cursor;

    REPLACE INTO user_engagement_report         (user_id, username,
posts_counted, total_points, best_post_id,          best_post_points,
average_points, generated_at)     VALUES
        (p_user_id, v_username, v_count, v_total, v_best_post, v_best_points,
         ROUND(IF(v_count = 0, 0, v_total / v_count), 1), NOW());

    SELECT * FROM user_engagement_report WHERE user_id = p_user_id;
END;
```

Procedure 2: GenerateTrendingPostReport(IN p_start DATETIME, IN p_end DATETIME, IN p_limit INT)

Scenario	The editorial team wants the trending list for a chosen week or month.
Definition	Trending means a post *received activity* in the window, not that it was *published* in the window. An older post people are reacting to this week is trending; a new post nobody touched is not.
Cursor use	The cursor selects posts with an interaction or comment BETWEEN the two dates. Each is scored with CalculateWindowEngagement and inserted into trending_report only if it earned points. After the cursor closes, the procedure returns the list ranked with RANK() OVER.
Cost	The row-by-row version takes a few hundred milliseconds where the set-based Query lab query takes about 60 ms. Both return the same top 20, which makes a good consistency check to demonstrate.
Demonstrated at	Procedures and triggers page, button GenerateTrendingPostReport

```
CREATE PROCEDURE GenerateTrendingPostReport(IN p_start DATETIME, IN p_end DATETIME, IN p_limit INT)
BEGIN
    DECLARE v_done     INT DEFAULT 0;
    DECLARE v_post_id  INT;
    DECLARE v_author   VARCHAR(30);
    DECLARE v_topic    VARCHAR(60);
    DECLARE v_points   DECIMAL(10, 1);

    DECLARE window_cursor CURSOR FOR
        SELECT p.post_id, u.username, p.topic
          FROM posts p
          JOIN users u ON u.user_id = p.user_id
```

```
        WHERE EXISTS (SELECT 1 FROM interactions i
                      WHERE i.post_id = p.post_id
                        AND i.created_at BETWEEN p_start AND p_end)
           OR EXISTS (SELECT 1 FROM comments c
                      WHERE c.post_id = p.post_id
                        AND c.created_at BETWEEN p_start AND p_end)
        ORDER BY p.post_id;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    DELETE FROM trending_report;

    OPEN window_cursor;     scan_loop: LOOP         FETCH
window_cursor INTO v_post_id, v_author, v_topic;
        IF v_done = 1 THEN
            LEAVE scan_loop;
        END IF;         SET v_points = CalculateWindowEngagement(v_post_id,

p_start, p_end);

        IF v_points > 0 THEN
            INSERT INTO trending_report
                (post_id, author, topic, points, window_start, window_end, generated_at)
            VALUES                (v_post_id, v_author, v_topic, v_points,
p_start, p_end, NOW());
        END IF;
    END LOOP;
    CLOSE window_cursor;

    SELECT post_id, author, topic, points AS points_in_window,
           RANK() OVER (ORDER BY points DESC) AS trend_rank
      FROM trending_report
     ORDER BY points DESC
LIMIT p_limit;
END;
```

5.2 Functions

Function 1: CalculatePostEngagement(p_post_id INT) RETURNS DECIMAL(10,1)

Scenario	The scoring rule appears in the dashboard, the leaderboard, both procedures and several queries. Putting it in one function means the weights change in one place.
Behaviour	Returns the lifetime weighted score of one post: view 0.2, like 1, save 2, share 3, plus 2 per comment. It returns the same number as v_post_engagement.
Use	SELECT CalculatePostEngagement(2341);

```
CREATE FUNCTION CalculatePostEngagement(p_post_id INT)
RETURNS DECIMAL(10, 1)
READS SQL DATA
BEGIN
    DECLARE v_points DECIMAL(10, 1);

    SELECT COALESCE(SUM(CASE i.interaction_type
                          WHEN 'view'  THEN 0.2
                          WHEN 'like'  THEN 1.0
                          WHEN 'save'  THEN 2.0
                          WHEN 'share' THEN 3.0
                          ELSE 0 END), 0)
      INTO v_points
      FROM interactions i
     WHERE i.post_id = p_post_id;     SET v_points = v_points + (SELECT COUNT(*) * 2.0 FROM

comments c WHERE c.post_id = p_post_id);

    RETURN ROUND(v_points, 1);
END;
```

Function 2: CalculateUserEngagement(p_user_id INT) RETURNS DECIMAL(12,1)

Scenario	A creator-level score is needed for the report procedure and for the dashboard.
Behaviour	Totals the points across everything a creator has published by calling Function 1 for each of their posts. It also shows one stored function calling another.

```
CREATE FUNCTION CalculateUserEngagement(p_user_id INT)
RETURNS DECIMAL(12, 1)
READS SQL DATA
```

```
BEGIN
    DECLARE v_total DECIMAL(12, 1) DEFAULT 0;

    SELECT COALESCE(SUM(CalculatePostEngagement(p.post_id)), 0)
      INTO v_total
      FROM posts p
     WHERE p.user_id = p_user_id;

    RETURN ROUND(v_total, 1);
END;
```

Function 3: CalculateWindowEngagement(p_post_id INT, p_start DATETIME, p_end DATETIME) RETURNS DECIMAL(10,1)

Scenario	Lifetime score and trending score are different questions. A March post with 300 points earned in March is not trending today.
Behaviour	The same weighted score, counting only interactions and comments dated between two moments. GenerateTrendingPostReport uses it so the procedure and the Query lab mean the same thing by trending.

```
CREATE FUNCTION CalculateWindowEngagement(p_post_id INT, p_start DATETIME, p_end DATETIME)
RETURNS DECIMAL(10, 1)
READS SQL DATA
BEGIN
    DECLARE v_points DECIMAL(10, 1);

    SELECT COALESCE(SUM(CASE i.interaction_type
                          WHEN 'view'  THEN 0.2
                          WHEN 'like'  THEN 1.0
                          WHEN 'save'  THEN 2.0
                          WHEN 'share' THEN 3.0
                          ELSE 0 END), 0)
      INTO v_points
      FROM interactions i
     WHERE i.post_id = p_post_id
       AND i.created_at BETWEEN p_start AND p_end;

    SET v_points = v_points + (SELECT COUNT(*) * 2.0
                               FROM comments c
                              WHERE c.post_id = p_post_id
                                AND c.created_at BETWEEN p_start AND p_end);

    RETURN ROUND(v_points, 1);
END;
```

5.3 Triggers

Trigger 1: trg_no_self_engagement (BEFORE INSERT ON interactions)

Scenario	Nobody should boost their own post by liking, saving, sharing or viewing it.
Why a trigger	The rule compares the incoming user_id with posts.user_id, a value in another table. A CHECK constraint cannot read another table, so a trigger is the right tool.
Effect	On violation it raises SIGNAL SQLSTATE '45000' and the insert is rejected. The dashboard shows the message when you "like it as the author".

```
CREATE TRIGGER trg_no_self_engagement
BEFORE INSERT ON interactions
FOR EACH ROW
BEGIN
    DECLARE v_author INT;     SELECT user_id INTO v_author FROM posts

WHERE post_id = NEW.post_id;

    IF v_author = NEW.user_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'A user cannot interact with their own post';
    END IF;
END;
```

Trigger 2: trg_interaction_audit (AFTER INSERT ON interactions)

Scenario	Every engagement event should leave a record, separate from the live table.

Effect	Each new interaction is copied into interaction_audit with its type, user, post and timestamp. It records events, not counts, so no totals are duplicated and nothing can drift out of sync.
Load order	The triggers are created after the bulk data load, so the 177,000 generated rows do not each write an audit row.

```
CREATE TRIGGER trg_interaction_audit
AFTER INSERT ON interactions
FOR EACH ROW
BEGIN
    INSERT INTO interaction_audit
        (interaction_id, post_id, user_id, interaction_type, action, recorded_at)
    VALUES
        (NEW.interaction_id, NEW.post_id, NEW.user_id, NEW.interaction_type, 'insert', NOW());
END;
```

Trigger 3: trg_post_delete_audit (AFTER DELETE ON posts)

Scenario	Deleting a post cascades to its comments and interactions, so those rows vanish. The team wants a record of what was removed and when.
Effect	The deleted post's id, author and topic are written to post_audit, shown in the dashboard's audit trail after a delete.

```
CREATE TRIGGER trg_post_delete_audit
AFTER DELETE ON posts
FOR EACH ROW
BEGIN
    INSERT INTO post_audit (post_id, user_id, topic, action, recorded_at)
VALUES (OLD.post_id, OLD.user_id, OLD.topic, 'delete', NOW());
END;
```

6. Front-end and Back-end Connectivity

6.1 Choice of front-end

The front-end is a single-page dashboard in **HTML5, CSS3 and vanilla JavaScript**, served by Express. Charts are drawn as inline SVG. There is no build step and no CDN dependency, so it runs the same on any machine.

Why this choice satisfies the project:

- **It shows the database, not itself.** Every number on every page is fetched from the API when the page loads, and the SQL behind it is shown next to the result.
- **It connects to MySQL through a back-end.** The browser talks only to the Express API. The API holds the connection pool and the SQL. The browser never receives database credentials.
- **It is easy to learn and to explain in a viva.** Plain JavaScript with fetch() has no framework to learn, so the team can concentrate on the SQL.
- **It is portable.** `docker compose up` starts MySQL and the app together, or the app can point at XAMPP, WAMP or a standalone MySQL server through a .env file.

6.2 Back-end connectivity with MySQL

Concern	How it is handled
Driver	mysql2 (promise API) with a connection pool of up to 5 connections
Configuration	Host, port, user, password and database name come from environment variables or a single DATABASE_URL, so no credentials sit in the code
First run	The server checks for an empty database and creates tables, views, indexes, data and stored programs itself; the page shows a setup screen while it works
Query safety	Catalog queries are looked up by id; search endpoints use ? placeholders; sort columns come from an allow-list

SQL console	One SELECT or WITH only, write keywords blocked, read-only transaction, execution timeout, outer LIMIT 200
Stored programs	Called with CALL and SELECT from the API, for example CALL GenerateUserEngagementReport(?)

6.3 Dashboard pages and the API behind them

Page	What it does	API endpoints
Overview	Daily activity chart, totals, top posts, top creators, interaction mix	GET /api/overview
Explore data	Search users and posts; open a user profile	GET /api/users, /api/users/:id, /api/posts
Query lab	Run each of the 14 analytical queries and see the SQL	GET /api/queries, POST /api/queries/:id/run
SQL console	Run a read-only SELECT of your own	POST /api/sql
Index lab	Drop and create the project indexes, benchmark a query, read EXPLAIN	GET /api/indexes, POST /api/indexes/:action, POST /api/benchmark
Procedures & triggers	Call functions and cursor procedures; create, like and delete a post; read the audit trail	GET /api/programs, /api/audit; POST /api/functions, /api/procedures, /api/posts, /api/interactions; DELETE /api/posts/:id
Schema	Columns, constraints and row counts read from information_schema	GET /api/schema

6.4 Learning resources

Official documentation used for the front-end and back-end: MDN Web Docs (HTML, CSS, JavaScript, fetch API), the Node.js documentation, the Express guide, the mysql2 documentation, and the MySQL 8.0 Reference Manual (window functions, CTEs, stored programs, EXPLAIN).